

AIMS System - Detailed Class Design Documentation

1. Design Class: PlaceOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentCart	Cart	null	Reference to the customer's current shopping cart
2	currentOrder	Order	null	Order being created in the current process
3	deliveryInfo	DeliveryInfo	null	Delivery information for the current order
4	currentInvoice	Invoice	null	Invoice generated for the current order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	requestToPlaceOrder	boolean	Initiates the order placement process with items from cart
2	checkAvailabilityOfChosenItems	boolean	Verifies if all items in cart are available in sufficient quantity
3	Order	Order	Creates a new order with items from the cart
4	DeliveryForm	DeliveryForm	Creates and returns a delivery form for customer input
5	submitDeliveryInfo	boolean	Processes the delivery information submitted by customer
6	checkValidityOfDeliveryInfo	boolean	Validates the delivery information format and completeness
7	DeliveryInfo	DeliveryInfo	Creates a delivery info object from validated input
8	calculateShippingFee	float	Calculates shipping fee based on product weight and delivery location
9	Invoice	Invoice	Creates an invoice with order and delivery information
10	placeOrder	boolean	Finalizes the order placement after payment is processed
11	save	void	Saves the confirmed order to the database
12	empty	void	Clears the cart after successful order placement
13	sendSuccessOrderEmail	boolean	Sends order confirmation email to customer
14	OrderInfoScreen	OrderInfoScreen	Creates a screen displaying order and transaction details
15	placeRushOrder	boolean	Verify if there exists any products for rush delivery from cart

Parameter:

- **requestToPlaceOrder(Cart): boolean**
 - **cartObj:** The shopping cart containing products to be ordered
 - **Return:** true if all products are available and order can proceed, false otherwise
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **cart:** The shopping cart to check availability for
 - **Return:** true if all items are available in sufficient quantity, false otherwise
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information fields (name, address, etc.)
 - **Return:** true if delivery information is valid, false otherwise
- **checkValidityOfDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information to validate

- **Return:** true if all required fields are valid, false otherwise
- **calculateShippingFee(deliveryInfo: DeliveryInfo, order: Order): float**
 - **deliveryInfo:** Customer's delivery information including location
 - **order:** The order containing products with weight information
 - **Return:** The calculated shipping fee in the local currency
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **transactionInfo:** Payment transaction details
 - **Return:** true if order was successfully placed, false otherwise
- **placeRushOrder(province: String, district: String): boolean**
 - **province:** entered by a Administrator
 - **district:** entered by a Administrator
 - **Return:** true if there exists any cart items for rush delivery

Exception:

- **requestToPlaceOrder(Cart): boolean**
 - **EmptyCartException:** Thrown if cart is empty
 - **InvalidCartException:** Thrown if cart contains invalid items
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **DatabaseConnectionException:** Thrown if unable to check product availability
 - **InsufficientStockException:** Details which products have insufficient stock
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **InvalidDeliveryInfoException:** Details which fields are invalid or missing
- **requestToPlaceOrder(Cart): boolean**
 - **UnsupportedLocationException:** Thrown if delivery location is not supported
 - **InvalidWeightException:** Thrown if product weight data is missing or invalid

Method:

- **requestToPlaceOrder(cartObj: Cart): boolean**
 - **Purpose:** Initiates the order placement process by checking product availability
 - **Processing:**
 - Validates that the cart is not empty
 - For each item in the cart:
 - Calls checkProductAvailability on the Product object
 - If any product is unavailable, returns false and provides details on unavailable items
 - If all products are available, creates a new Order object
 - Creates and returns a DeliveryForm for customer input
 - **Return:** true if order process can continue, false if any products are unavailable
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **Purpose:** Processes and validates delivery information provided by customer
 - **Processing:**
 - Validates all required delivery information fields
 - Creates a DeliveryInfo object with the validated information
 - Calculates shipping fee based on product weights and delivery location
 - Creates an Invoice with order details, products, VAT, and shipping fee
 - Returns an InvoiceScreen for customer review
 - **Return:** true if delivery information is valid and processing continues, false otherwise
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **Purpose:** Finalizes order placement after successful payment
 - **Processing:**
 - Saves the order to the database with "Pending Processing" status
 - Empties the cart
 - Sends order confirmation email to customer
 - Creates and displays OrderInfoScreen with order and transaction details
 - **Return:** true if order was successfully placed, false if any step fails

State:

- Before requestToPlaceOrder is called, currentCart contains the customer's selected products
- After successful requestToPlaceOrder, currentOrder contains a new Order object
- After submitDeliveryInfo, deliveryInfo contains validated delivery information and currentInvoice contains the generated invoice
- After successful placeOrder, the order is saved to the database, the cart is emptied, and confirmation email is sent